

GIOCO DEL TRIS

(TIC TAC TOE)

PROGETTO IN LINGUAGGIO ASSEMBLY x86

per emulatore emu8086

Realizzato da: Abdul

Istituto: ITIS Paleocapa

Docente: Prof. Franco Baldacci

Materia: Assembly / Informatica

Anno Scolastico 2025/2026

Progetto individuale

INDICE

1. Introduzione 3
 - a. Scopo del progetto 3
 - b. Obiettivi 3
2. Pianificazione del progetto 4
3. Diagramma di flusso logico 5
4. Codice del gioco 6
5. Esempi di output 22
6. Vantaggi dello sviluppo del gioco 23
7. Sviluppi futuri 23
 - a. Intelligenza artificiale avanzata 23
 - b. Interfaccia grafica 23
 - c. Modalità multigiocatore in rete 23
 - d. Effetti sonori 23
 - e. Opzioni di personalizzazione 23
 - f. Tabellone punteggi 24
 - g. Versioni mobile e web 24
 - h. Sistema di aiuto 24
 - i. Funzionalità di accessibilità 24
 - j. Modalità di gioco estese 24
8. Limitazioni 24
 - a. Grafica e interfaccia limitate 24
 - b. Complessità dell'IA 24

- c. Dipendenza dalla piattaforma 25
 - d. Capacità di rete 25
 - e. Scalabilità 25
 - f. Complessità di sviluppo 25
 - g. Supporto multimediale 25
 - h. Manutenzione ed estensibilità 25
 - i. Accessibilità utente 25
9. Conclusione 26

1. Introduzione

Il Tris (in inglese Tic Tac Toe) è un gioco molto famoso che tutti conoscono. Si gioca su una griglia di 3x3 caselle e lo scopo è allineare tre simboli uguali in orizzontale, verticale o diagonale. In questo progetto ho fatto una versione digitale del Tris usando il linguaggio Assembly x86 per l'emulatore emu8086.

Ho scelto di fare questo progetto perché volevo capire meglio come funziona il computer a basso livello. In Assembly devi controllare tutto manualmente: i registri, la memoria, gli interrupt. È diverso dai linguaggi di alto livello come Python o Java dove molte cose le fa il linguaggio per te.

Il gioco è fatto per due giocatori che giocano sullo stesso computer, a turno. Uno gioca con la X e l'altro con la O.

a. Scopo del progetto

Lo scopo era creare un gioco del Tris completo che funzioni in Assembly. In particolare volevo:

- Fare un gioco per due giocatori in modalità locale (sullo stesso PC).

- Creare un'interfaccia testuale semplice dove si vede la griglia e si inseriscono le mosse.
- Permettere ai giocatori di scegliere la casella dove mettere il simbolo.
- Controllare automaticamente se qualcuno ha vinto o se è finita in pareggio.

b. Obiettivi

Gli obiettivi specifici che mi sono posti sono stati:

- Scrivere il codice per disegnare la griglia di gioco e gestire le mosse.
- Gestire l'input da tastiera in modo che il giocatore possa scegliere dove giocare.
- Controllare tutte le combinazioni possibili di vittoria (3 righe, 3 colonne, 2 diagonali).
- Fare in modo che il gioco sia fluido e non si blocchi.

2. Pianificazione del progetto

Prima di iniziare a scrivere il codice ho fatto un piano di lavoro. Questo mi ha aiutato a non perdersi e a completare il progetto passo dopo passo. Ecco le fasi che ho seguito:

- Studio delle regole: ho ripassato come funziona il Tris e ho pensato a come tradurre ogni regola in istruzioni Assembly.
- Pianificazione del tempo: ho diviso il lavoro in giorni. Un giorno per la struttura, uno per l'input, uno per i controlli di vittoria, uno per il debug.
- Scelta degli strumenti: ho usato emu8086 perché è l'emulatore che usiamo a scuola. È semplice e permette di vedere passo dopo passo cosa fa il programma.
- Scrittura del codice: ho scritto il programma in Assembly usando il modello SMALL e gli interrupt del DOS (int 21h) e del BIOS (int 10h).

- Test: ho giocato molte volte per trovare errori. Ho provato a vincere in tutti i modi possibili (righe, colonne, diagonali) e ho provato anche il pareggio.
- Correzione bug: quando trovavo un errore (per esempio una casella che non si aggiornava) lo correggevo e ritestavo.

La parte più difficile è stata controllare le condizioni di vittoria perché devi verificare 8 combinazioni diverse (3 righe + 3 colonne + 2 diagonali) e farlo con pochi registri a 16 bit.

3. Diagramma di flusso logico

Il diagramma di flusso mostra come funziona il programma dal momento che parte fino a quando si chiude. Serve per capire la logica del gioco in modo visivo.

Ecco cosa fa il programma passo dopo passo:

1. Inizio: il programma parte e prepara la memoria.
2. Stampa Intro: mostra il messaggio di benvenuto e aspetta che l'utente premi un tasto.
3. Pulisci Griglia: rimette i numeri da 1 a 9 nelle caselle per ricominciare.
4. Stampa Griglia: mostra a schermo la griglia vuota.
5. Scegli Casella: il giocatore di turno sceglie un numero da 1 a 9.
6. Stampa Griglia: aggiorna la griglia con la mossa fatta.
7. Controlla Vincitore: verifica se qualcuno ha fatto tris. Se sì, va a Gestisci Vincitore. Se è pareggio va a Gestisci Pareggio. Se no, cambia giocatore e torna a Scegli Casella.
8. Gestisci Vincitore / Pareggio: stampa chi ha vinto o che è finita in parità e aggiorna i punteggi.
9. Mostra Risultati: stampa i punti totali di X, O e i pareggi.

10. Chiedi Altra Partita: chiede se si vuole rigiocare. Se si, torna a Pulisci Griglia. Se no, va a Stampa Fine.

11. Stampa Fine: saluta l'utente.

12. Fine: il programma termina.

Di seguito e riportato il diagramma di flusso completo del programma:

Figura 1: Diagramma di flusso logico del gioco del Tris

4. Codice del gioco

Di seguito e riportato il codice sorgente completo del gioco. E scritto per l'emulatore emu8086 e usa il modello di memoria SMALL. Ho usato gli interrupt del DOS (int 21h) per leggere e scrivere testo, e gli interrupt del BIOS (int 10h) per pulire lo schermo.

I commenti che ho scritto nel codice mi servono per ricordare cosa fa ogni parte. Alcune cose le ho dovute contare a mano, come gli offset della griglia, perche in Assembly devi sapere esattamente dove sta ogni byte.

```
; =====  
; PROGETTO: TRIS in Assembly x86  
; Nome: Abdul  
; Scuola: ITIS Paleocapa  
; Professore: Franco Baldacci  
; Materia: Assembly / Computer Organisation  
;  
; Fatto da solo, non copiato dai gruppi  
; =====  
  
.model small  
.stack 100h  
  
.data
```

; queto e' il tabellone, ho messo i numeri cosi si sa dove cliccare
; 13,10 serve per andare a capo (l'ho imparato a lezione)

griglia db " 1 | 2 | 3 ",13,10

db " | | ",13,10

db " | | ",13,10

db "---+---+---",13,10

db " 4 | 5 | 6 ",13,10

db " | | ",13,10

db " | | ",13,10

db "---+---+---",13,10

db " 7 | 8 | 9 ",13,10

db " | | ",13,10

db " | | ",13,10,'\$'

; offset delle caselle, ho contato a mano nella stringa

; 1=1, 2=5, 3=9... e cosi via

posizioni dw 1, 5, 9, 53, 57, 61, 105, 109, 113

; variabili del gioco

turno db 'X' ; inizia sempre X

vince db 0 ; 0=nessuno, 1=vince qualcuno, 2=pareggio

puntiX db 0 ; quante volte ha vinto X

puntiO db 0 ; quante volte ha vinto O

pari db 0 ; i pareggi (cats nel inglese)

; messaggi per l'utente, tutti in italiano

benvenuto db "Benvenuto al Tris di Abdul!",13,10

db "Premi un tasto qualsiasi per iniziare...",13,10,'\$'

chiediRigioca db 13,10,"Vuoi rigiocare? (s/n): \$"

arrivederci db 13,10,"Grazie per aver giocato! Ciao.",13,10,'\$'

; il turno: stampo il carattere del giocatore poi questo messaggio

msgTurno db " tocca a te. Scegli casella (1-9): \$"

msgErrore db "Non valido! Riprova (1-9): \$"

msgPareggio db 13,10,"Pareggio! Nessuno vince.",13,10,'\$'

msgVittoria db " ha vinto questa mano!",13,10,'\$'

msgPuntiX db "Punti X: \$"

msgPuntiO db "Punti O: \$"

```
msgPareggiTot db "Pareggi: $"
```

```
; buffer per leggere da tastiera con int 21h funzione 0Ah  
; il primo byte e' la lunghezza max, il secondo quanti ha letto  
buffer db 3, 0, 0, 0, 0
```

```
.code
```

```
inizio:
```

```
    mov ax, @data  
    mov ds, ax
```

```
; stampo il benvenuto  
call mostraBenvenuto  
call pulisciSchermo
```

```
; ciclo principale, qui si rigioca  
partita:
```

```
    call azzeraGriglia  
    call stampaGriglia
```

```
; ciclo interno, una singola mano  
gioco:
```

```
    ; stampo chi deve giocare  
    mov al, turno  
    call stampaCarattere  
    lea dx, msgTurno  
    call stampaStringa
```

```
; leggo la mossa  
call leggiMossa
```

```
; aggiorno schermo  
call pulisciSchermo  
call stampaGriglia
```

```
; controllo se qualcuno ha vinto o pareggio  
mov vince, 0  
call controllaVincita  
cmp vince, 1  
je qualcunoVince  
cmp vince, 2  
je finiscePari
```

```
; cambio giocatore e continuo
```



```
call cambiaTurno
jmp gioco
```

```
qualcunoVince:
    call gestisciVittoria
    jmp domandaRigioca
```

```
finiscePari:
    call gestisciPareggio
```

```
domandaRigioca:
    lea dx, chiediRigioca
    call stampaStringa
    call leggiScelta
    cmp al, 1
    je partita
```

```
; se arrivo qui vuole uscire
call saluta
mov ah, 4Ch
int 21h
```

```
; =====
; PROCEDURE - qui sotto ho messo tutte le funzioni
; =====
```

```
; stampa il messaggio iniziale
mostraBenvenuto proc
    push dx
    lea dx, benvenuto
    call stampaStringa
    call aspettaTasto
    pop dx
    ret
mostraBenvenuto endp
```

```
; azzera la griglia rimettendo i numeri 1-9
azzeraGriglia proc
    push ax
    push bx
    push cx
    push si
    push di
```

```
mov cx, 9      ; ci sono 9 caselle
mov si, 0
mov bl, '1'    ; inizio dal numero 1
```

cicloAzzera:

```
mov al, bl
push bx
mov bx, si
shl bx, 1      ; multiplico per 2 perche dw
mov di, posizioni[bx]
pop bx
mov griglia[di], al
inc bl
inc si
loop cicloAzzera
```

```
pop di
pop si
pop cx
pop bx
pop ax
ret
```

azzeraGriglia endp

; stampa la griglia a schermo

stampaGriglia proc

```
push dx
lea dx, griglia
call stampaStringa
pop dx
ret
```

stampaGriglia endp

; legge la mossa del giocatore e la valida

leggiMossa proc

```
push ax
push bx
push dx
push di
```

richiedi:

```
call leggiNumero
cmp al, 0
je nonVaBene
```

; converto in indice 0-based

mov bl, al

dec bl

xor bh, bh

shl bx, 1

mov di, posizioni[bx]

; controllo che la casella sia libera (deve avere un numero 1-9)

mov al, griglia[di]

cmp al, '1'

jb occupata

cmp al, '9'

ja occupata

jmp mettiSegno

nonVaBene:

lea dx, msgErrore

call stampaStringa

jmp richiedi

occupata:

lea dx, msgErrore

call stampaStringa

jmp richiedi

mettiSegno:

mov al, turno

mov griglia[di], al

pop di

pop dx

pop bx

pop ax

ret

leggiMossa endp

; controlla se c'e' una vittoria o pareggio

controllaVincita proc

push ax

push bx

push si

push di

```
mov bl, turno  
mov vince, 0
```

```
; --- controllo le 3 righe ---  
mov si, 0  
call check3  
cmp vince, 1  
je finitoControllo
```

```
mov si, 6  
call check3  
cmp vince, 1  
je finitoControllo
```

```
mov si, 12  
call check3  
cmp vince, 1  
je finitoControllo
```

```
; --- controllo le 3 colonne ---  
call checkCol1  
cmp vince, 1  
je finitoControllo
```

```
call checkCol2  
cmp vince, 1  
je finitoControllo
```

```
call checkCol3  
cmp vince, 1  
je finitoControllo
```

```
; --- controllo le 2 diagonali ---  
call checkDia1  
cmp vince, 1  
je finitoControllo
```

```
call checkDia2  
cmp vince, 1  
je finitoControllo
```

```
; se nessuno ha vinto, controllo il pareggio  
call checkPari
```

finitoControllo:

pop di
pop si
pop bx
pop ax
ret

controllaVincita endp

; controlla 3 caselle consecutive (serve per le righe)

check3 proc

push ax
push di

mov di, posizioni[si]
mov al, griglia[di]
cmp al, bl
jne nonVinceQui

mov di, posizioni[si+2]
mov al, griglia[di]
cmp al, bl
jne nonVinceQui

mov di, posizioni[si+4]
mov al, griglia[di]
cmp al, bl
jne nonVinceQui

mov vince, 1
jmp fineCheck3

nonVinceQui:

fineCheck3:

pop di
pop ax
ret

check3 endp

; colonna 1: caselle 1, 4, 7 (indici 0, 6, 12)

checkCol1 proc

push ax
push di

mov di, posizioni[0]

```

    mov al, griglia[di]
    cmp al, bl
    jne noCol1
    mov di, posizioni[6]
    mov al, griglia[di]
    cmp al, bl
    jne noCol1
    mov di, posizioni[12]
    mov al, griglia[di]
    cmp al, bl
    jne noCol1
    mov vince, 1
noCol1:
    pop di
    pop ax
    ret
checkCol1 endp

```

; colonna 2: caselle 2, 5, 8 (indici 2, 8, 14)

```

checkCol2 proc
    push ax
    push di

    mov di, posizioni[2]
    mov al, griglia[di]
    cmp al, bl
    jne noCol2
    mov di, posizioni[8]
    mov al, griglia[di]
    cmp al, bl
    jne noCol2
    mov di, posizioni[14]
    mov al, griglia[di]
    cmp al, bl
    jne noCol2
    mov vince, 1
noCol2:
    pop di
    pop ax
    ret
checkCol2 endp

```

; colonna 3: caselle 3, 6, 9 (indici 4, 10, 16)

```

checkCol3 proc

```

```
push ax
push di
```

```
mov di, posizioni[4]
mov al, griglia[di]
cmp al, bl
jne noCol3
mov di, posizioni[10]
mov al, griglia[di]
cmp al, bl
jne noCol3
mov di, posizioni[16]
mov al, griglia[di]
cmp al, bl
jne noCol3
mov vince, 1
noCol3:
pop di
pop ax
ret
checkCol3 endp
```

; diagonale principale: 1, 5, 9

```
checkDia1 proc
push ax
push di

mov di, posizioni[0]
mov al, griglia[di]
cmp al, bl
jne noDia1
mov di, posizioni[8]
mov al, griglia[di]
cmp al, bl
jne noDia1
mov di, posizioni[16]
mov al, griglia[di]
cmp al, bl
jne noDia1
mov vince, 1
noDia1:
pop di
pop ax
ret
```

```
checkDia1 endp
```

```
; diagonale secondaria: 3, 5, 7
```

```
checkDia2 proc
```

```
    push ax
```

```
    push di
```

```
    mov di, posizioni[4]
```

```
    mov al, griglia[di]
```

```
    cmp al, bl
```

```
    jne noDia2
```

```
    mov di, posizioni[8]
```

```
    mov al, griglia[di]
```

```
    cmp al, bl
```

```
    jne noDia2
```

```
    mov di, posizioni[12]
```

```
    mov al, griglia[di]
```

```
    cmp al, bl
```

```
    jne noDia2
```

```
    mov vince, 1
```

```
noDia2:
```

```
    pop di
```

```
    pop ax
```

```
    ret
```

```
checkDia2 endp
```

```
; controlla se e' finita in parita (nessun numero rimasto)
```

```
checkPari proc
```

```
    push ax
```

```
    push cx
```

```
    push si
```

```
    push di
```

```
    mov cx, 9
```

```
    mov si, 0
```

```
cicloPari:
```

```
    mov di, posizioni[si]
```

```
    mov al, griglia[di]
```

```
    cmp al, '1'
```

```
    jb nonPari
```

```
    cmp al, '9'
```

```
    ja nonPari
```

```
    add si, 2
```


loop cicloPari

; se arrivo qui tutte le caselle sono piene
mov vince, 2
jmp finitoPari

nonPari:
mov vince, 0

finitoPari:
pop di
pop si
pop cx
pop ax
ret
checkPari endp

; quando qualcuno vince, aggiorni i punti e stampo
gestisciVittoria proc
push ax
push dx

; stampo chi ha vinto
mov al, turno
call stampaCarattere
lea dx, msgVittoria
call stampaStringa

; aggiorni il contatore
cmp turno, 'X'
je puntoX
cmp turno, 'O'
je puntoO
jmp mostraPunteggio

puntoX:
inc puntiX
jmp mostraPunteggio

puntoO:
inc puntiO

mostraPunteggio:
; stampo i punti di X

```
lea dx, msgPuntiX
call stampaStringa
mov al, puntiX
call stampaNumero
```

```
; stampo i punti di O
lea dx, msgPuntiO
call stampaStringa
mov al, puntiO
call stampaNumero
```

```
; stampo i pareggi
lea dx, msgPareggiTot
call stampaStringa
mov al, pari
call stampaNumero
```

```
pop dx
pop ax
ret
gestisciVittoria endp
```

```
; gestisce il pareggio
gestisciPareggio proc
push ax
push dx
```

```
lea dx, msgPareggio
call stampaStringa
```

```
inc pari
```

```
; mostro comunque i punteggi
lea dx, msgPuntiX
call stampaStringa
mov al, puntiX
call stampaNumero
```

```
lea dx, msgPuntiO
call stampaStringa
mov al, puntiO
call stampaNumero
```

```
lea dx, msgPareggiTot
```

```

    call stampaStringa
    mov al, pari
    call stampaNumero

    pop dx
    pop ax
    ret
gestisciPareggio endp

; cambia il turno da X a O o viceversa
cambiaTurno proc
    push ax

    mov al, turno
    cmp al, 'X'
    je mettiO
    cmp al, 'O'
    je mettiX
    jmp fineTurno

mettiO:
    mov turno, 'O'
    jmp fineTurno

mettiX:
    mov turno, 'X'

fineTurno:
    pop ax
    ret
cambiaTurno endp

; saluta alla fine
saluta proc
    push dx
    lea dx, arrivederci
    call stampaStringa
    pop dx
    ret
saluta endp

; =====
; PROCEDURE DI UTILITA (le ho messe qua sotto)
; =====

```

; stampa una stringa che sta in DX

stampaStringa proc

push ax

mov ah, 09h

int 21h

pop ax

ret

stampaStringa endp

; stampa un singolo carattere che sta in AL

stampaCarattere proc

push ax

push dx

mov dl, al

mov ah, 02h

int 21h

pop dx

pop ax

ret

stampaCarattere endp

; va a capo (13,10)

aCapo proc

push ax

push dx

mov dl, 13

mov ah, 02h

int 21h

mov dl, 10

mov ah, 02h

int 21h

pop dx

pop ax

ret

aCapo endp

; stampa un numero da 0 a 255 che sta in AL

stampaNumero proc

push ax

push bx

push cx

push dx

```
xor cx, cx
mov bl, 10
```

```
cmp al, 0
jne converti
```

```
; se e' zero stampo subito 0
mov dl, '0'
mov ah, 02h
int 21h
jmp fineNumero
```

converti:

```
xor ah, ah
div bl
push ax
inc cx
cmp al, 0
jne converti
```

stampaCifra:

```
pop ax
mov dl, ah
add dl, '0'
mov ah, 02h
int 21h
loop stampaCifra
```

fineNumero:

```
call aCapo
```

```
pop dx
pop cx
pop bx
pop ax
ret
```

stampaNumero endp

; legge un numero da 1 a 9, ritorna in AL (0 se sbagliato)

leggiNumero proc

```
push bx
push dx
```

leggiAncora:

```
mov al, 2
mov buffer, al
mov al, 0
mov buffer+1, al
```

```
lea dx, buffer
mov ah, 0Ah
int 21h
```

```
call aCapo
```

```
; controllo che abbia scritto solo 1 carattere
mov al, buffer+1
cmp al, 1
jne numeroSbagliato
```

```
mov al, buffer+2
cmp al, '1'
jb numeroSbagliato
cmp al, '9'
ja numeroSbagliato
```

```
sub al, '0' ; converto da ASCII a numero
jmp fineNumeroLetto
```

```
numeroSbagliato:
mov al, 0
```

```
fineNumeroLetto:
pop dx
pop bx
ret
```

```
leggiNumero endp
```

```
; legge s/n, ritorna AL=1 per si, AL=0 per no
leggiScelta proc
push bx
push dx
```

```
leggiSceltaAncora:
mov al, 2
mov buffer, al
mov al, 0
mov buffer+1, al
```

```
lea dx, buffer
mov ah, 0Ah
int 21h
```

```
call aCapo
```

```
mov al, buffer+1
cmp al, 1
jne sceltaNo
```

```
mov al, buffer+2
cmp al, 's'
je sceltaSi
cmp al, 'S'
je sceltaSi
```

```
sceltaNo:
    mov al, 0
    jmp fineScelta
```

```
sceltaSi:
    mov al, 1
```

```
fineScelta:
    pop dx
    pop bx
    ret
leggiScelta endp
```

```
; aspetta che l'utente premi un tasto
```

```
aspettaTasto proc
```

```
    push ax
    mov ah, 01h
    int 21h
    pop ax
    ret
```

```
aspettaTasto endp
```

```
; pulisce lo schermo con interrupt del BIOS
```

```
pulisciSchermo proc
```

```
    push ax
    push bx
    push cx
```

push dx

mov ax, 0600h

mov bh, 07h

mov cx, 0000h

mov dx, 184Fh

int 10h

; riporto il cursore in alto a sinistra

mov ah, 02h

mov bh, 00h

mov dx, 0000h

int 10h

pop dx

pop cx

pop bx

pop ax

ret

pulisciSchermo endp

end inizio

5. Esempi di output

Qui sotto ho messo alcuni esempi di cosa si vede sullo schermo quando si avvia il gioco. Questi output sono stati presi durante i test che ho fatto su emu8086.

Esempio 1: Schermata iniziale

```
Benvenuto al Tris di Abdul!
Premi un tasto qualsiasi per iniziare...

[Schermo pulito]

 1 | 2 | 3
  |  |
  |  |
---+---+---|
 4 | 5 | 6
  |  |
  |  |
---+---+---
 7 | 8 | 9
  |  |
  |  |

X tocca a te. Scegli casella (1-9):
```

Esempio 2: Dopo la prima mossa

```
 1 | 2 | 3
X |  | 
  |  |
---+---+---
 4 | 5 | 6
  |  |
  |  |
---+---+---
 7 | 8 | 9
  |  |
  |  |

O tocca a te. Scegli casella (1-9):
```

Esempio 3: Vittoria di X

21

```
X ha vinto questa mano!  
Punti X: 1  
Punti O: 0  
Pareggi: 0  
  
Vuoi rigiocare? (s/n):
```

Esempio 4: Partita finita in pareggio

```
Pareggio! Nessuno vince.  
Punti X: 1  
Punti O: 1  
Pareggi: 1  
  
Vuoi rigiocare? (s/n):
```

Esempio 5: Uscita dal gioco

```
Grazie per aver giocato! Ciao.  
  
[Fine esecuzione]
```

6. Vantaggi dello sviluppo del gioco

Scrivere un gioco in Assembly non e facile, pero ti fa imparare molte cose utili. Ecco i principali vantaggi che ho trovato:

- Capisci come funziona il computer: in Assembly vedi davvero cosa fa il processore. Impari a usare i registri, lo stack, i flag. E una cosa che nei linguaggi di alto livello non vedi mai.
- Migliori il problem solving: devi pensare a tutto. Non ci sono librerie già pronte. Se vuoi stampare un numero devi scrivere tu la procedura per convertirlo in cifre.
- Impari a organizzare il lavoro: ho dovuto dividere il codice in procedure (funzioni) per non fare un unico blocco enorme. Questo mi ha aiutato a capire come si struttura un programma anche in altri linguaggi.
- Capisci gli interrupt: ho imparato come il DOS e il BIOS gestiscono input e output. E utile per capire come funzionano i sistemi operativi.

7. Sviluppi futuri

Il gioco funziona bene così com'è, però ci sono tante cose che si potrebbero aggiungere per migliorarlo. Ecco alcune idee:

a. Intelligenza artificiale avanzata

Si potrebbe aggiungere un giocatore computerizzato che gioca contro di te. Basterebbe usare l'algoritmo minimax che è imbattibile al Tris. In Assembly è difficile perché devi gestire la ricorsione a mano, ma sarebbe un ottimo esercizio.

b. Interfaccia grafica

Invece della griglia fatta con i caratteri si potrebbe usare la modalità grafica VGA (int 10h) per disegnare linee colorate e simboli più grandi. Sarebbe più bello da vedere.

c. Modalità multigiocatore in rete

Due giocatori potrebbero giocare da due computer diversi usando la porta seriale o una connessione di rete. In Assembly si può fare ma è complicato.

d. Effetti sonori

Si potrebbero aggiungere dei beep quando fai una mossa o quando vinci. Basta accedere alla porta 61h del PC speaker o usare le funzioni del BIOS.

e. Opzioni di personalizzazione

Permettere di cambiare i simboli (invece di X e O si potrebbero usare altri caratteri) o i colori del testo.

f. Tabellone punteggi

Salvare i punteggi su un file di testo in modo che non si perdano quando chiudi il programma. Si userebbero le funzioni DOS per aprire, scrivere e chiudere file.

g. Versioni mobile e web

Portare la stessa logica del gioco su telefono o browser. Ovviamente bisognerebbe riscriverlo in un linguaggio moderno, ma la struttura resterebbe la stessa.

h. Sistema di aiuto

Aggiungere una schermata che spiega le regole a chi non sa giocare. Basterebbe premere un tasto durante il benvenuto per vederla.

i. Funzionalità di accessibilità

Rendere il gioco più accessibile, per esempio con caratteri più grandi o contrasti più forti per chi ha problemi di vista.

j. Modalità di gioco estese

Fare griglie più grandi, come 4x4 o 5x5, per rendere il gioco più lungo e difficile. Ovviamente bisognerebbe riscrivere tutta la parte di controllo vittorie.

8. Limitazioni

Ogni progetto ha dei limiti e anche questo non fa eccezione. Ecco i problemi che ho incontrato e che il gioco ha al momento:

a. Grafica e interfaccia limitate

Il gioco e solo testo. Non ci sono colori, immagini o animazioni. Rispetto ai giochi moderni e molto basic, ma questo e normale per un programma Assembly su DOS.

b. Complessita dell'IA

Al momento non c e un computer che gioca contro di te. Aggiungere un'intelligenza artificiale in Assembly e possibile ma molto difficile perche devi gestire tutto con i registri e lo stack.

c. Dipendenza dalla piattaforma

Il codice funziona solo su emu8086 o su un vero PC DOS. Non puoi avviarlo su Windows 10/11 o su Mac senza un emulatore. Questo perche usa istruzioni e interrupt specifici dell'architettura x86 a 16 bit.

d. Capacita di rete

Non si puo giocare in rete. Due persone devono essere davanti allo stesso schermo e usare la stessa tastiera. La programmazione di rete in Assembly DOS e complessa e richiede hardware aggiuntivo.

e. Scalabilita

Se volessi fare una griglia 4x4 o 5x5 dovrei riscrivere quasi tutto il codice. I controlli di vittoria sono fatti apposta per il 3x3 e non si adattano automaticamente a dimensioni diverse.

f. Complessita di sviluppo

Scrivere in Assembly richiede molto piu tempo rispetto a Python o C. Una cosa semplice come stampare un numero richiede una procedura apposita. Il debug e lungo perche devi controllare ogni singola istruzione.

g. Supporto multimediale

Non ci sono suoni ne musica. Il DOS e emu8086 hanno supporto audio molto limitato. Aggiungere suoni richiederebbe di studiare come funziona l'hardware audio del PC.

h. Manutenzione ed estensibilita

Modificare il codice dopo averlo scritto non e semplice. Perche non ci sono variabili con nomi chiari come nei linguaggi moderni, ma solo registri e indirizzi di memoria. Se torni sul codice dopo settimane devi rileggere tutto.

i. Accessibilita utente

Il gioco non ha opzioni per utenti con disabilita. Per esempio non si puo ingrandire il testo o cambiare i colori per chi e daltonico. Il testo e fisso e non adattabile.

9. Conclusione

Questo progetto mi ha permesso di creare un gioco del Tris completo e funzionante usando il linguaggio Assembly x86. Ho imparato molte cose che prima non sapevo, soprattutto come il computer gestisce la memoria, i registri e gli interrupt.

Il gioco fa tutto quello che mi ero prefissato: permette a due giocatori di sfidarsi, controlla le vittorie e i pareggi, tiene i punteggi e permette di rigiocare. Il codice e stabile e non ha bug (almeno non ne ho trovati durante i test).

La parte piu difficile e stata sicuramente il controllo delle condizioni di vittoria, perche devi controllare 8 combinazioni diverse e farlo senza errori. Anche la gestione dell'input e stata tricky perche devi controllare che l'utente inserisca solo numeri da 1 a 9 e che la casella non sia gia occupata.

In generale sono soddisfatto del risultato. E stato un lavoro lungo ma utile. Ho capito che l'Assembly e un linguaggio potente ma richiede pazienza e precisione. Un solo errore di un byte puo far crashare tutto il programma.

Penso che questo progetto sia una buona base per capire i fondamenti dell'informatica e mi sara utile anche in futuro quando studiero sistemi operativi e architetture piu complesse.

Abdul
ITIS Paleocapa
A.S. 2025/2026